

IA Décision séquentielle pour un agent autonome

CM2: Apprentissage par renforcement

Laëtitia Matignon



Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base
- 5 AR passif
- 6 AR actif
- 7 Généralisation
 - Fusion d'états
 - Généralisation
 - Approximate Q-learning

Différents types d'apprentissage

	Supervisé	Non supervisé	Par renforcement
Apprend Information Méthode			

Différents types d'apprentissage

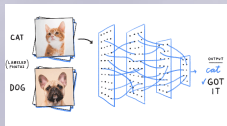
	Supervisé	Non supervisé	Par renforcement
Apprend Information Méthode	relations sortie désirée par instruction		

Apprentissage supervisé

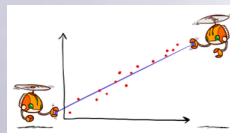
- Ensemble d'exemples (X_k, Y_k) où $Y = f(X)$ avec f fonction cible (inconnue) à apprendre.
- Application : classification, régression ...



Ensemble d'entraînement
labellisé



Classification



Régression (approximation
de fonction)

L'apprentissage c'est quoi ?

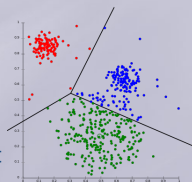
	Supervisé	Non supervisé	Par renforcement
Apprend	relations	structures ou patterns	
Information	sortie désirée	rien	
Méthode	par instruction	par observation	

Apprentissage non supervisé

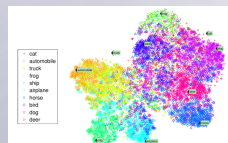
- Identifier des structures dans des données $(X_k)_k$ (ex. des classes)
- Regrouper les données selon des critères de ressemblance inconnus *a priori*
- Application : clustering (agrégation), réduction de dimensions, détection d'anomalies, ...



Ensemble d'entraînement non labellisé



Clustering (k-means)



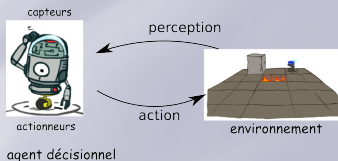
réduction de dimensions (Visualisation t-SNE)

L'apprentissage c'est quoi ?

	Supervisé	Non supervisé	Par renforcement
Apprend	relations	structures ou patterns	politique
Information	sortie désirée	rien	récompense
Méthode	par instruction	par observation	par essai/erreur

Apprentissage par renforcement

- Apprentissage basé sur l'interaction avec l'environnement :
l'apprenant est actif
- Apprendre par l'expérience en fonction des échecs ou succès constatés
(les **renforcements** ou **récompenses**).



Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base
- 5 AR passif
- 6 AR actif
- 7 Généralisation
 - Fusion d'états
 - Généralisation
 - Approximate Q-learning

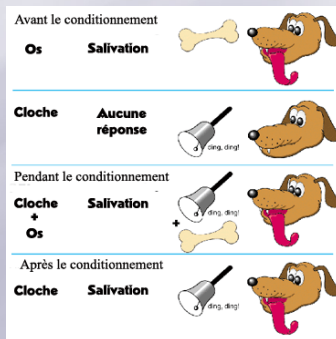
Origines de l'apprentissage par renforcement

C'est la rencontre de ...

Psychologie expérimentale

Modèles de conditionnement animal :

- conditionnement classique/pavlovien : association entre stimulus neutre (cloche) et inconditionnel (induit réflexe) donne stimulus conditionnel

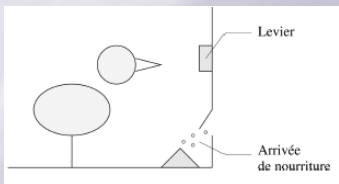


Chien de pavlov

Origines de l'apprentissage par renforcement

C'est la rencontre de ...

- ≡ Psychologie expérimentale : le courant behavioriste étudie comment une action peut être **renforcée**
renforcement = satisfaction, plaisir ou inconfort, douleur
- ≡ Modèles de conditionnement animal :
 - conditionnement opérant : apprendre par essais-erreurs une action conditionnée par une situation
 - La **récompense renforce** l'action "presser le bouton"
 - Modifier la récompense modifie le comportement **appris**



Loi de l'effet ([Thorndike,1911]) :

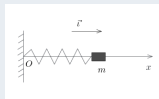
*Une action suivie d'un stimulus agréable sera **renforcée** et une action suivie d'un stimulus désagréable tendra à diminuer.*

FIGURE – Boite de Skinner

Origines de l'apprentissage par renforcement

C'est la rencontre de ...

- Psychologie expérimentale
- Théorie du contrôle optimal : **concevoir des contrôleurs** capables de **maximiser un critère** de performance d'un système dynamique
renforcement = critère à maximiser.



- Formalisation : Processus décisionnel markovien

L'objectif est de reproduire le conditionnement animal pour le contrôle de systèmes.

Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification**
- 4 Notions de base
- 5 AR passif
- 6 AR actif
- 7 Généralisation
 - Fusion d'états
 - Généralisation
 - Approximate Q-learning

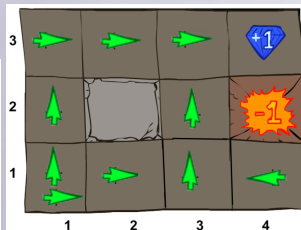
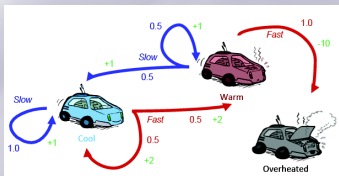
Rappels : Planification (*offline* MDP)

Planification

- Modèle MDP $\langle S, A, T, R \rangle$ **complètement connu**
- Solution au MDP : une politique π
- Objectif : trouver une politique optimale π^*

π^* donne pour tout état, la ou les action(s) permettant de maximiser les récompenses que l'on espère obtenir à travers la séquence d'états futurs.

π^* maximise le critère γ pondéré : $G_t = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots = \sum_{t=0}^{\infty} \gamma^t r_{t+1}$



Rappels : Planification (*offline* MDP)

Planification

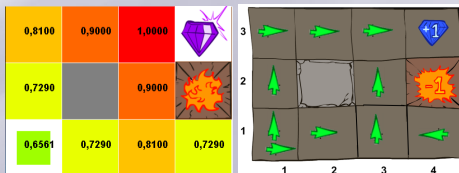
≡ Objectif : trouver une politique optimale π^*

≡ Moyen : fonction de valeur $V : S \rightarrow \mathbb{R}$

La valeur $V^\pi(s)$ d'un état s est la **somme des récompenses espérées** si on suit π depuis s : $V^\pi(s) = E\{G^\gamma | \pi\} = E\{\sum_{t=0}^{\infty} \gamma^t r_{t+1} | \pi, s_0 = s\}$

≡ Algorithme *value iteration* : calcul itératif de V^* :

$$\forall s \in S \quad V_{k+1}(s) = \max_{a \in A} \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \gamma V_k^\pi(s')]$$

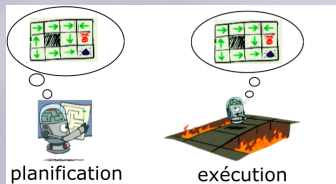


Fonction de valeur V associée à la politique π dans un environnement déterministe

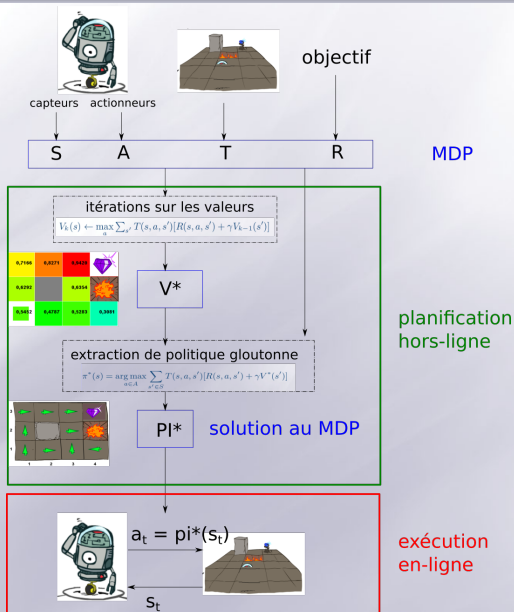
Rappels : Planification (*offline* MDP)

Planification **hors-ligne**

- Objectif : trouver une politique optimale π^*
- Moyen : fonction de valeur $V : S \rightarrow \mathbb{R}$
- Algorithme *value iteration* : calcul itératif de V^*
- Problème de **planification** dans l'incertain :
 - Phase **hors ligne** : calcul itératif de V^* **sans être placé dans l'environnement** (*value iteration*)
 - Puis phase d'exécution : exécution de la politique **dans l'environnement**



Rappels : Planification (*offline* MDP)



Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base**
- 5 AR passif
- 6 AR actif
- 7 Généralisation
 - Fusion d'états
 - Généralisation
 - Approximate Q-learning

Apprentissage par renforcement

MDP partiellement connu

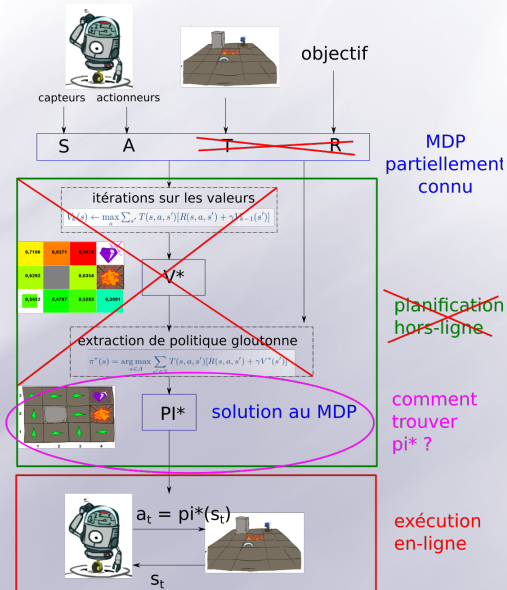
- ≡ Modèle MDP $\langle S, A, T, R \rangle$
- ≡ Objectif : trouver la politique optimale π^*
- ≡ Nouvelle hypothèse : **T et R inconnus de l'agent**
 - L'agent ne connaît pas à l'avance les effets de ses actions
 - L'agent ne connaît pas à l'avance les actions et/ou états récompensés

→ Impossible de planifier !



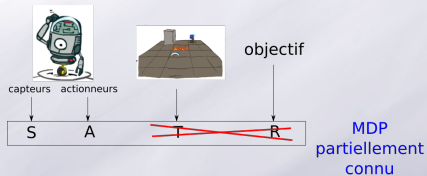
Éléments du MDP connus par l'agent dans le cas de l'apprentissage par renforcement

AR vs Planification

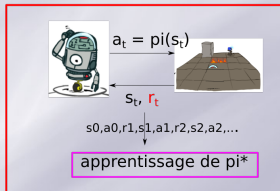


AR vs Planification

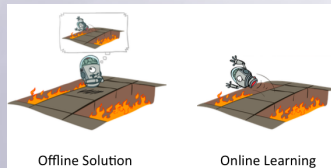
Comment l'agent peut apprendre π^* avec un MDP partiellement connu ?



- Interaction avec l'environnement : apprentissage en ligne
- Essais et erreurs (renforcement)

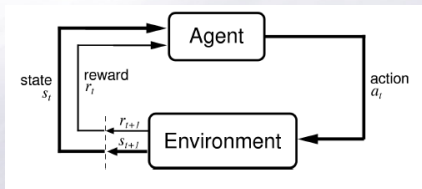


apprentissage en-ligne



Planification vs. Apprentissage

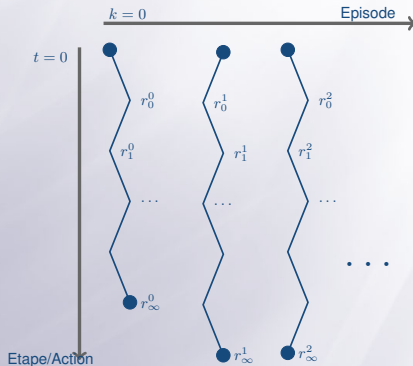
Temporalité en Apprentissage par Renforcement



Agent plongé dans un MDP sans connaissance de R et T .

Double temporalité :

- Une étape t = une action
- Un épisode k = ensemble d'étapes/actions de l'état initial à un état absorbant



Exemple et courbes d'apprentissage



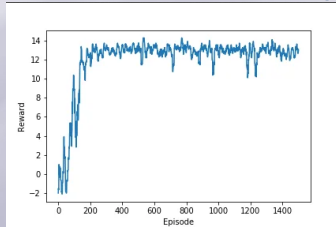
Episode initial



* Episodes d'entraîne-
ment



Episode après appren-
tissage



Exemple d'une courbe d'apprentissage : somme des récompenses par épisode

Types d'Apprentissage par Renforcement

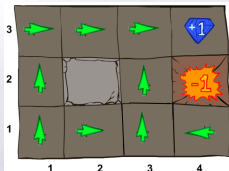
- ≡ AR indirect (*model-based*) :
 - apprendre R et T par interactions (forme d'apprentissage supervisé)
 - s'appuyer sur cette connaissance pour calculer π^* avec des méthodes de planification
- ≡ AR direct (*model-free*) : construire π^* en apprenant des fonctions spécifiques mais sans apprentissage direct de R et T

Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base
- 5 AR passif**
- 6 AR actif
- 7 Généralisation
 - Fusion d'états
 - Généralisation
 - Approximate Q-learning

AR passif

AR passif : Tâche simplifiée d'évaluation d'une politique fixée



Pour une π fixée
(T et R inconnus)

0,8100	0,9000	1,0000	
0,7290		0,9000	
0,6561	0,7290	0,8100	0,7290

V^π (ex. d'un env
déterministe)

π fixée $\rightarrow$ Comment évaluer V^π ?

Pour chaque état s , on veut évaluer :

$$V^\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right\}$$

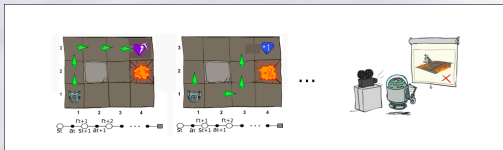
La valeur $V^\pi(s)$ d'un état s selon une politique π est l'espérance de récompenses cumulées G si l'agent démarre dans l'état s et suit la politique π

AR passif

π fixée $\rightarrow$ Comment évaluer V^π ?

Pour chaque état s , on veut évaluer : $V^\pi(s) = E_\pi \{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} | s_0 = s \}$

- l'agent **interagit directement** avec l'environnement
- il réalise plusieurs **épisodes** en suivant la politique fixée π



- Episode 1 : $\langle 1, 1 \rangle, UP, 0, \langle 1, 2 \rangle, UP, 0, \langle 1, 3 \rangle, RIGHT, 0, \langle 2, 3 \rangle, RIGHT, 0, \langle 3, 3 \rangle, RIGHT, 1$
- Episode 2 : $\langle 1, 1 \rangle, UP, 0, \langle 2, 1 \rangle, RIGHT, 0, \langle 3, 1 \rangle, UP, 0, \langle 3, 2 \rangle, UP, -1$
- ...
- Episode k : $s_{0,k}, a_{0,k}, r_{1,k}, s_{1,k}, a_{1,k}, r_{2,k}, \dots$

3 algorithmes pour évaluer V^π à partir des épisodes

AR passif : Algorithme 1

π fixée $\rightarrow$ Comment évaluer V^π ?

Pour chaque état s , on veut évaluer : $V^\pi(s) = E_\pi\{\sum_{t=0}^{\infty} \gamma^t r_{t+1} | s_0 = s\}$

- ≡ Episode 1 : $\langle 1, 1 \rangle, UP, 0, \langle 1, 2 \rangle, UP, 0, \langle 1, 3 \rangle, RIGHT, 0, \langle 2, 3 \rangle, RIGHT, 0, \langle 3, 3 \rangle, RIGHT, 1$
- ≡ Episode 2 : $\langle 1, 1 \rangle, UP, 0, \langle 2, 1 \rangle, RIGHT, 0, \langle 3, 1 \rangle, UP, 0, \langle 3, 2 \rangle, UP, -1$
- ≡ Episode k : $s_{0,k}, a_{0,k}, r_{1,k}, s_{1,k}, a_{1,k}, r_{2,k}, \dots$

Algorithme 1

La valeur d'un état s $V^\pi(s)$ est évaluée comme la **moyenne sur tous les épisodes**, des récompenses cumulées obtenues à partir de s

$V^\pi(\langle 1, 1 \rangle)? V^\pi(\langle 3, 1 \rangle)?$

AR passif : Algorithme 1

Algorithme 1

La valeur d'un état s $V^\pi(s)$ est évaluée comme la **moyenne sur tous les épisodes**, des récompenses cumulées obtenues **à partir de s**

≡ Episode 1 : $\langle 1, 1 \rangle, UP, 0, \langle 1, 2 \rangle, UP, 0, \langle 1, 3 \rangle, RIGHT, 0, \langle 2, 3 \rangle, RIGHT, 0, \langle 3, 3 \rangle, RIGHT, 1$

≡ Episode 2 : $\langle 1, 1 \rangle, UP, 0, \langle 2, 1 \rangle, RIGHT, 0, \langle 3, 1 \rangle, UP, 0, \langle 3, 2 \rangle, UP, -1$

≡ Episode k : $s_{0,k}, a_{0,k}, r_{1,k}, s_{1,k}, a_{1,k}, r_{2,k}, \dots$

≡ Dans épisode 1 : $v^\pi(\langle 1, 1 \rangle) = \gamma^0 * 0 + \gamma^1 * 0 + \gamma^2 * 0 + \gamma^3 * 0 + \gamma^4 * 1$

≡ Dans épisode 2 : $v^\pi(\langle 1, 1 \rangle) = \gamma^0 * 0 + \gamma^1 * 0 + \gamma^2 * 0 + \gamma^3 * -1$

≡ $V^\pi(\langle 1, 1 \rangle) = \frac{\gamma^4 - \gamma^3}{2}$

AR passif : Algorithme 1

Algorithme 1

La valeur d'un état s $V^\pi(s)$ est évaluée comme la **moyenne sur tous les épisodes**, des récompenses cumulées obtenues **à partir de s**

- ≡ la valeur d'un état s $V^\pi(s)$ est évaluée comme la **moyenne sur tous les épisodes**, des récompenses cumulées obtenues **à partir de s**
- ≡ Episode 1 : $\langle 1, 1 \rangle, UP, 0, \langle 1, 2 \rangle, UP, 0, \langle 1, 3 \rangle, RIGHT, 0, \langle 2, 3 \rangle, RIGHT, 0, \langle 3, 3 \rangle, RIGHT, 1$
- ≡ Episode 2 : $\langle 1, 1 \rangle, UP, 0, \langle 2, 1 \rangle, RIGHT, 0, \langle 3, 1 \rangle, UP, 0, \langle 3, 2 \rangle, UP, -1$
- ≡ Episode k : $s_{0,k}, a_{0,k}, r_{1,k}, s_{1,k}, a_{1,k}, r_{2,k}, \dots$

$$\equiv V^\pi(\langle 3, 1 \rangle) = \gamma^0 * 0 + \gamma^1 * -1 = -\gamma$$

AR passif : Algorithme 1 : Monte-Carlo Prediction

AR passif : Tâche simplifiée d'évaluation d'une politique fixée

π fixée $\rightarrow$ Comment évaluer V^π ?

Pour chaque état s , on veut évaluer :

$$V^\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right\}$$

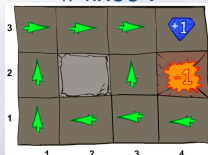
Algorithme 1 : Monte-Carlo Prediction

- ≡ on stocke des épisodes (trajectoires) complets réalisés dans l'environnement en suivant la politique π
- ≡ on utilise une **moyenne** pour estimer la **valeur** $V^\pi(s)$ de chaque état :
 - la **valeur** d'un état s **pour un épisode** k est appelé **échantillon** : somme des récompenses obtenues depuis s dans cet épisode :

$$v(s_k) = r_{1,k} + \gamma r_{2,k} + \gamma^2 r_{3,k} + \dots$$
 - si s est visité plusieurs fois dans un épisode, on garde la première visite (*first-visit*) ou on moyenne toutes les visites de l'épisode (*every-visit*)
 - la **valeur** d'un état s est la **moyenne** de ses **échantillons** sur tous les épisodes : $V^\pi(s) = \frac{1}{n} \sum_{i=1}^n \{v(s_i) \mid s_i = s\}$

Algorithme Monte-Carlo Prediction : Exemple

π fixée :



$\gamma = 1$, $R(*, *, s) = -0.1$

Episode $k = 1$:

t=0 (1, 1) NORD -0.1

t=1 (1, 2) NORD -0.1

t=2 (1, 2) NORD -0.1

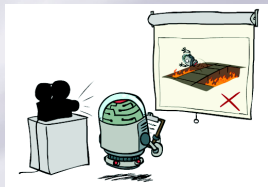
t=3 (1, 3) EST -0.1

t=4 (2, 3) EST -0.1

t=5 (3, 3) EST -0.1

t=6 (3, 2) NORD -0.1

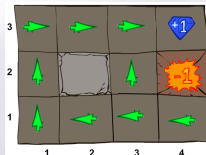
t=7 (3, 3) EST +1



$v(< 2, 3 >) = ?$ $v(< 3, 3 >) = ?$

Algorithme Monte-Carlo Prediction : Exemple

π fixée :



$\gamma = 1, R(*, *, s) = -0.1$

Episode $k = 1$:

t=0 (1, 1) NORD -0.1

t=1 (1, 2) NORD -0.1

t=2 (1, 2) NORD -0.1

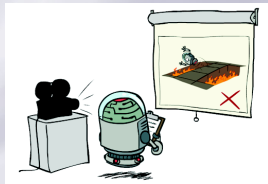
t=3 (1, 3) EST -0.1

t=4 (2, 3) EST -0.1

t=5 (3, 3) EST -0.1

t=6 (3, 2) NORD -0.1

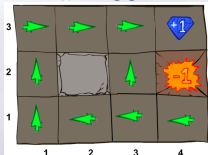
t=7 (3, 3) EST +1



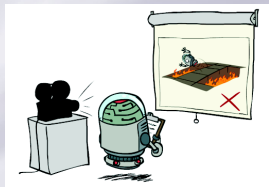
- valeur de état $\langle 2, 3 \rangle$ pour cet épisode : $v(\langle 2, 3 \rangle) = 1 - 0.1 \times 3$
- valeur de état $\langle 3, 3 \rangle$ pour cet épisode : $v(\langle 3, 3 \rangle) = 1 - 0.1 \times 2$ et $v(\langle 3, 3 \rangle) = 1$ (*every visit* : état visité 2 fois dans l'épisode)

Algorithme Monte-Carlo Prediction : Exemple

π fixée :



$\gamma = 1, R(*, *, s) = -0.1$



Episode $k = 1$:

t=0 (1,1) NORD -0.1

t=1 (1,2) NORD -0.1

t=2 (1,2) NORD -0.1

t=3 (1,3) EST -0.1

t=4 (2,3) EST -0.1

t=5 (3,3) EST -0.1

t=6 (3,2) NORD -0.1

t=7 (3,3) EST +1

Episode $k = 2$:

t=0 (1,1) NORD -0.1

t=1 (1,2) NORD -0.1

t=2 (1,3) EST -0.1

t=3 (2,3) EST -0.1

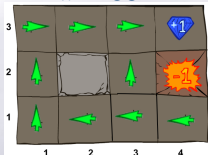
t=4 (3,3) EST -0.1

t=5 (3,2) NORD -1

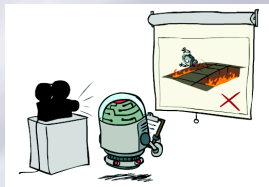
$v(< 2, 3 >) = ? \quad v(< 3, 3 >) = ?$

Algorithme Monte-Carlo Prediction : Exemple

π fixée :



$\gamma = 1, R(*, *, s) = -0.1$



Episode $k = 1$:

t=0 (1, 1) NORD -0.1

t=1 (1, 2) NORD -0.1

t=2 (1, 2) NORD -0.1

t=3 (1, 3) EST -0.1

t=4 (2, 3) EST -0.1

t=5 (3, 3) EST -0.1

t=6 (3, 2) NORD -0.1

t=7 (3, 3) EST +1

Episode $k = 2$:

t=0 (1, 1) NORD -0.1

t=1 (1, 2) NORD -0.1

t=2 (1, 3) EST -0.1

t=3 (2, 3) EST -0.1

t=4 (3, 3) EST -0.1

t=5 (3, 2) NORD -1

- = valeur de état $\langle 2, 3 \rangle$ pour épisode 2 : $v(\langle 2, 3 \rangle) = -1 - 0.1 \times 2$
- = valeur de état $\langle 3, 3 \rangle$ pour épisode 2 : $v(\langle 3, 3 \rangle) = -1 - 0.1$

Algorithme Monte-Carlo Prediction : Exemple



$$\gamma = 1, R(*, *, s) = -0.1$$

Episode $k = 1$:

t=0 (1, 1) NORD -0.1

t=1 (1, 2) NORD -0.1

t=2 (1, 2) NORD -0.1

t=3 (1, 3) EST -0.1

t=4 (2, 3) EST -0.1

t=5 (3, 3) EST -0.1

t=6 (3, 2) NORD -0.1

t=7 (3, 3) EST +1

Episode $k = 2$:

t=0 (1, 1) NORD -0.1

t=1 (1, 2) NORD -0.1

t=2 (1, 3) EST -0.1

t=3 (2, 3) EST -0.1

t=4 (3, 3) EST -0.1

t=5 (3, 2) NORD -1

Si on ne considère que 2 épisodes, la valeur obtenue pour ces 2 états est alors :

$$V(< 2, 3 >) = \frac{1}{2} \times [(1 - 0.1 \times 3) + (-1 - 0.1 \times 2)] = -0.25$$

$$V(< 3, 3 >) = \frac{1}{3} \times [(1 - 0.1 \times 2) + (1) + (-1 - 0.1)] = 0.23$$

AR passif : Algorithme 1 : Monte-Carlo Prediction

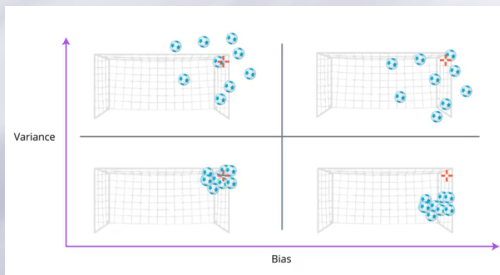
Algorithme 1 : Monte-Carlo Prediction

Intérêts/Inconvénients :

- + : apprentissage en ligne, à partir d'expériences, sans modèle
- - : nécessite de conserver tous les épisodes
- - : convergence très lente, nécessite beaucoup d'épisodes car il faut visiter tous les états un grand nombre de fois (car stochasticité)
- - : problème pour les épisodes de taille infinie
- - : forte variance : la valeur (échantillon) d'un état peut être très différente d'un épisode à un autre (car stochasticité)
- + : pas de biais : les valeurs (échantillon) ne sont pas biaisées car issues d'une interaction avec l'environnement

Rappels : biais et variance en apprentissage

- biais : erreur provenant d'hypothèses erronées ou de mauvaises valeurs pour les échantillons ; mauvaise estimation de la moyenne
- variance : erreur due à la sensibilité aux petites fluctuations des échantillons



AR passif : Algorithme Monte-Carlo Prediction

Algorithme 1 : Monte-Carlo Prediction

Intérêts/Inconvénients :

- ≡ + : apprentissage en ligne, à partir d'expériences, sans modèle
- ≡ - : nécessite de conserver tous les épisodes
- ≡ - : convergence très lente, nécessite beaucoup d'épisodes car il faut visiter tous les états un grand nombre de fois (car stochasticité)
- ≡ - : problème pour les épisodes de taille infinie
- ≡ - : forte variance : la valeur (échantillon) d'un état peut être très différente d'un épisode à un autre (car stochasticité)
- ≡ + : pas de biais : les valeurs (échantillon) ne sont pas biaisées car issues d'une interaction avec l'environnement

Exercice TD

Amélioration : calcul incrémental de la moyenne !

Calcul incrémental d'une moyenne

Calculer la **moyenne** V des 3 valeurs obtenues sur 3 épisodes $v = \{1, 4, 7\}$?

- ☰ Méthode non incrémentale : utilise tous les épisodes pour calculer la moyenne

$$V = \frac{1 + 4 + 7}{3} = 4$$

- ☰ Méthode incrémentale : **utilise uniquement l'épisode courant** pour mettre à jour la moyenne

Episode	$k = 1$	$k = 2$	$k = 3$
Valeur (échantillon) de s dans l'épisode	$v(s) = 1$	$v(s) = 4$	$v(s) = 7$
Valeur estimée de s (moyenne incrémentale)	$V(s) = 1$	$V(s) = 2.5$ $V(s) = 1 + \frac{1}{2}[4 - 1]$	$V(s) = 4$ $V(s) = 2.5 + \frac{1}{3}[7 - 2.5]$

$$V = V + \frac{1}{k}[v - V]$$

A chaque épisode, on calcule une **nouvelle estimation de $V(s)$** , à partir de **l'ancienne estimation de $V(s)$** , de **l'échantillon le plus récent $v(s)$** , et du nombre total d'échantillons.

Calcul incrémental de moyenne : détails

- Comment calculer la moyenne V des 3 valeurs $v_i = \{1, 4, 7\}$?
- Méthode incrémentale : **Estimation incrémentale** de V en calculant V_k (*new estimation*) à partir de V_{k-1} (*old estimation*).
A chaque épisode k , on reçoit un nouvel échantillon v_k (pour un état s)

$$V_k = \frac{1}{k} \sum_{i=1}^k v_i \quad (1)$$

$$= \frac{1}{k} (v_k + \sum_{i=1}^{k-1} v_i) \quad (2)$$

$$= \frac{1}{k} (v_k + (k-1)V_{k-1} + V_{k-1} - V_{k-1}) \quad (3)$$

$$V_k = V_{k-1} + \frac{1}{k} [v_k - V_{k-1}] \quad (4)$$

new estimation = old estimation +

$\alpha_k [\text{new echantillon} - \text{old estimation}]$

Calcul incrémental de moyenne : intuition

A chaque épisode k , on reçoit un nouvel échantillon v_k , on met à jour l'évaluation de la moyenne V selon :

$$\underbrace{V_k}_{\text{nouvelle estimation}} = \underbrace{V_{k-1}}_{\text{ancienne estimation}} + \underbrace{\frac{1}{k}}_{\alpha_k} \left[\underbrace{v_k}_{\text{échantillon}} - \underbrace{V_{k-1}}_{\text{ancienne estimation}} \right]$$

δ

- ▮ δ erreur sur estimation
- ▮ Si $\delta > 0$, on augmente V car l'évaluation de notre moyenne est trop faible
- ▮ Si $\delta < 0$, on diminue V car l'évaluation de notre moyenne est trop élevée

Calcul incrémental de moyenne : rôle de α

A chaque épisode k , on reçoit un nouvel échantillon v_k , on met à jour l'évaluation de la moyenne V selon :

$$\underbrace{V_k}_{\text{nouvelle estimation}} = \underbrace{V_{k-1}}_{\text{ancienne estimation}} + \underbrace{\frac{1}{k}}_{\alpha_k} \left[\underbrace{v_k}_{\text{échantillon}} - \underbrace{V_{k-1}}_{\text{ancienne estimation}} \right]$$

δ

Avec $V_0 = 0$ et $k = 3$:

$$\begin{aligned} V_3 &= \underbrace{V_2} + \frac{1}{3}[v_3 - \underbrace{V_2}] \\ &= \underbrace{V_1} + \frac{1}{2}[v_2 - \underbrace{V_1}] + \frac{1}{3}[v_3 - V_1 - \frac{1}{2}[v_2 - \underbrace{V_1}]] \\ &= v_1 + \frac{1}{2}[v_2 - v_1] + \frac{1}{3}[v_3 - v_1 - \frac{1}{2}[v_2 - v_1]] \\ &= \frac{1}{3}v_1 + \frac{1}{3}v_2 + \frac{1}{3}v_3 \end{aligned} \tag{5}$$

α variable : les anciennes et nouvelles valeurs v_i sont pondérées de la même façon.

Calcul itératif de moyenne : rôle de α

A chaque épisode k , on reçoit un nouvel échantillon v_k , on met à jour l'évaluation de la moyenne V selon :

$$\underbrace{V_k}_{\text{nouvelle estimation}} = \underbrace{V_{k-1}}_{\text{ancienne estimation}} + \underbrace{\frac{1}{k}}_{\alpha_k} \left[\underbrace{v_k}_{\text{échantillon}} - \underbrace{V_{k-1}}_{\text{ancienne estimation}} \right]_{\delta}$$

Avec $V_0 = 0$ et $k = 3$, si on utilise α constant :

$$\begin{aligned} V_3 &= (1 - \alpha)V_2 + \alpha v_3 \\ &= (1 - \alpha)^2 v_1 + \alpha(1 - \alpha)v_2 + \alpha v_3 \end{aligned}$$

α **constant** : les anciennes et nouvelles valeurs v_i ne sont plus pondérées de la même façon : **oubli exponentiel des anciennes valeurs** v_i .

AR passif : Algorithme 2 : Incremental Monte-Carlo Prediction

AR passif : Tâche simplifiée d'évaluation d'une politique fixée

π fixée $\rightarrow$ Comment évaluer V^π ?

Pour chaque état s , on veut évaluer :

$$V^\pi(s) = E_\pi \left\{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s_0 = s \right\}$$

Algorithme 2 : Incremental Monte-Carlo Prediction

- on réalise des épisodes (trajectoires) complets réalisés dans l'environnement en suivant la politique π
- à chaque fin d'épisode, on met à jour les états visités pendant l'épisode :

$$\underbrace{V^\pi(s)}_{\text{nouvelle_estimation}} = \underbrace{V^\pi(s)}_{\text{ancienne_estimation}} + \alpha \left(\underbrace{v(s)}_{\text{echantillon}} - \underbrace{V^\pi(s)}_{\text{ancienne_estimation}} \right)$$

avec $v(s)$ échantillon pour cet épisode de la valeur de s :

$$v(s) = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$$

- coefficient d'apprentissage $\alpha \in [0; 1]$

Algorithme 2 : Incremental Monte-Carlo Prediction

Intérêts/Inconvénients :

- ≡ + : apprentissage en ligne, à partir d'expériences, sans modèle
- ≡ + : **nécessite de conserver un seul épisode (vs tous les épisodes pour MC Prediction)**
- ≡ - : convergence très lente, nécessite beaucoup d'épisodes car il faut visiter tous les états un grand nombre de fois (car stochasticité)
- ≡ - : problème pour les épisodes de taille infinie
- ≡ - : forte variance : la valeur (échantillon) d'un état peut être très différente d'un épisode à un autre (car stochasticité)
- ≡ + : pas de biais : les valeurs (échantillons) ne sont pas biaisées car issues d'une interaction avec l'environnement

Amélioration : calcul de l'échantillon à chaque action !

Calcul de l'échantillon à chaque action

Soit l'épisode : $s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, s_3 \dots$

On s'intéresse à une étape $\langle s_0, a_0, s_1, r_1 \rangle$: l'agent est dans s_0 , exécute l'action $a_0 = \pi(s_0)$, reçoit r_1 et arrive dans s_1 :

- jusqu'à maintenant on a calculé l'échantillon $v(s)$ avec la somme des récompenses obtenues depuis s dans l'épisode

Pour s_0 , on a $v(s_0) = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$

- la valeur de l'état d'arrivée s_1 est aussi une évaluation de la somme des récompenses obtenues depuis s_1 :

$V^\pi(s_1)$ évalue $r_2 + \gamma r_3 + \gamma^2 r_4 + \dots$

- on a donc :

$$\begin{aligned}
 v(s_0) &= r_1 + \gamma r_2 + \gamma^2 r_3 + \dots \\
 &= r_1 + \gamma \underbrace{(r_2 + \gamma r_3 + \dots)}_{V(s_1)} \\
 &= r_1 + \gamma V(s_1)
 \end{aligned}$$

Calcul de l'échantillon à chaque action

Soit l'épisode en suivant $\pi : s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, s_3 \dots$

On s'intéresse à une étape $\langle s_0, a_0, s_1, r_1 \rangle$: l'agent est dans s_0 , exécute l'action $a_0 = \pi(s_0)$, reçoit r_1 et arrive dans s_1 :

- ⌘ jusqu'à maintenant on a calculé l'échantillon $v(s)$ avec la somme des récompenses obtenues depuis s dans l'épisode
Pour s_0 , on a $v(s_0) = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$
- ⌘ la valeur de l'état d'arrivée s_1 est aussi une évaluation de la somme des récompenses obtenues depuis s_1 dans l'épisode :
 $V^\pi(s_1)$ évalue $r_2 + \gamma r_3 + \gamma^2 r_4 + \dots$
- ⌘ on a donc :

$$\begin{aligned}
 v(s_0) &= r_1 + \gamma r_2 + \gamma^2 r_3 + \dots \\
 &= r_1 + \gamma \underbrace{(r_2 + \gamma r_3 + \dots)}_{\simeq V^\pi(s_1)} \\
 &\simeq r_1 + \gamma V^\pi(s_1)
 \end{aligned}$$

Pour une étape $\langle s, a, s', r \rangle$, on peut prendre comme nouvel échantillon pour la moyenne incrémentale : $v(s) = r + \gamma V^\pi(s')$
 V^π étant la fonction que l'on est en train d'évaluer ...

AR passif : algorithme 3 : TD Prediction

AR passif : Tâche simplifiée d'évaluation d'une politique fixée

π fixée $\rightarrow$ Comment évaluer V^π ?

Pour chaque état s , on veut évaluer : $V^\pi(s) = E_\pi \{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} | s_0 = s \}$

Algorithme 3 : TD (*temporal difference*) Prediction

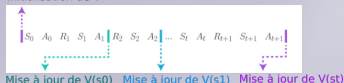
- on réalise des épisodes (trajectoires) complets réalisés dans l'environnement en suivant la politique π
- après chaque action exécutée par l'agent dans un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante à l'état s :

$$\underbrace{V^\pi(s)}_{\text{nouvelle_estimation}} = \underbrace{V^\pi(s)}_{\text{ancienne_estimation}} + \alpha \left(\underbrace{v(s)}_{\text{echantillon}} - \underbrace{V^\pi(s)}_{\text{ancienne_estimation}} \right)$$

avec $v(s)$ échantillon pour cette étape de la valeur de s : $v(s) = r + \gamma V^\pi(s')$

- l'échantillon utilise la valeur de l'état d'arrivée : *bootstrap*
- coefficient d'apprentissage $\alpha \in [0; 1]$

Initialisation de V



AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $< s, a = \pi(s), s', r >$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{new_estim} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{old_estim} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{new_echantillon}$$

Episode 2 :

t=0 (1, 1) NORD 0

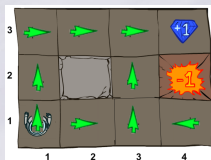
t=1 (1, 2) NORD 0

t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST 0

t=5 (3, 2) NORD -1



Episode 1 :

t=0 (1, 1) NORD 0

t=1 (1, 2) NORD 0

t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST +1

La politique à évaluer avec

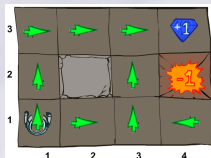
$\gamma = 1.0$ et $\alpha = 0.1$

Précisez pour chaque action/étape de chaque épisode, l'état mis à jour et sa nouvelle valeur.

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $< s, a = \pi(s), s', r >$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{new_estim} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{old_estim} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{new_echantillon}$$



Episode 1 :

t=0 (1, 1) NORD 0

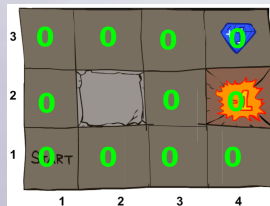
t=1 (1, 2) NORD 0

t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST +1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$

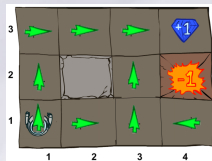


$V \text{ init à } 0 \forall s$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{\text{new_estim}} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{\text{old_estim}} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{\text{new_echantillon}}$$



Episode 1 :

t=0 (1, 1) NORD 0

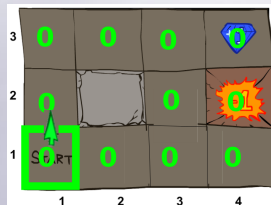
t=1 (1, 2) NORD 0

t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST +1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 1, $t = 0$,

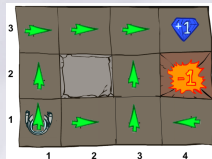
$s = \langle 1, 1 \rangle, s' = \langle 1, 2 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[0 + \gamma 0]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{new_estim} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{old_estim} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{new_echantillon}$$



Episode 1 :

t=0 (1,1) NORD 0

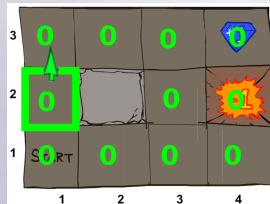
t=1 (1,2) NORD 0

t=2 (1,3) EST 0

t=3 (2,3) EST 0

t=4 (3,3) EST +1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 1, $t = 1$,

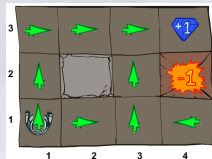
$s = \langle 1, 2 \rangle, s' = \langle 1, 3 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[0 + \gamma 0]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{new_estim} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{old_estim} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{new_echantillon}$$



Episode 1 :

t=0 (1, 1) NORD 0

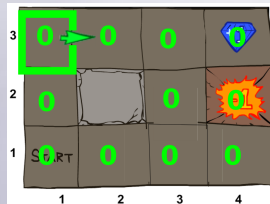
t=1 (1, 2) NORD 0

t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST +1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 1, $t = 2$,

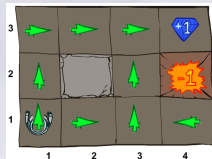
$s = \langle 1, 3 \rangle, s' = \langle 2, 3 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[0 + \gamma 0]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{new_estim} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{old_estim} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{new_echantillon}$$



Episode 1 :

t=0 (1,1) NORD 0

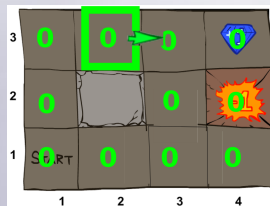
t=1 (1,2) NORD 0

t=2 (1,3) EST 0

t=3 (2,3) EST 0

t=4 (3,3) EST +1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 1, $t = 3$,

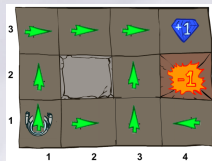
$s = \langle 2, 3 \rangle, s' = \langle 3, 3 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[0 + \gamma 0]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{\text{new_estim}} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{\text{old_estim}} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{\text{new_echantillon}}$$



Episode 1 :

$t=0$ (1,1) NORD 0

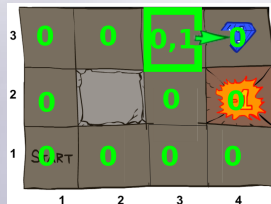
$t=1$ (1,2) NORD 0

$t=2$ (1,3) EST 0

$t=3$ (2,3) EST 0

$t=4$ (3,3) EST +1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 1, $t = 4$,

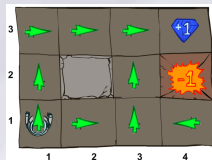
$s = \langle 3, 3 \rangle, s' = \langle 4, 3 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[1 + \gamma 0] = 0.1$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante à l'état s :

$$\underbrace{V^\pi(s)}_{new_estim} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{old_estim} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{new_echantillon}$$



Episode 2 :

$t=0$ (1, 1) NORD 0

$t=1$ (1, 2) NORD 0

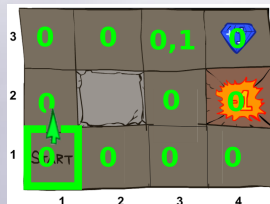
$t=2$ (1, 3) EST 0

$t=3$ (2, 3) EST 0

$t=4$ (3, 3) EST 0

$t=5$ (3, 2) NORD -1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 2, $t = 0$,

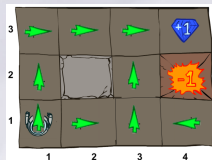
$s = \langle 1, 1 \rangle, s' = \langle 1, 2 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[0 + \gamma 0]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{\text{new_estim}} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{\text{old_estim}} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{\text{new_echantillon}}$$



Episode 2 :

t=0 (1, 1) NORD 0

t=1 (1, 2) NORD 0

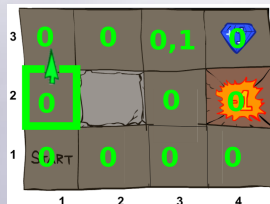
t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST 0

t=5 (3, 2) NORD -1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 2, $t = 1$,

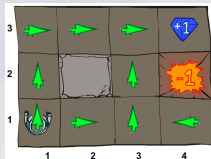
$s = \langle 1, 2 \rangle$, $s' = \langle 1, 3 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[0 + \gamma 0]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{new_estim} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{old_estim} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{new_echantillon}$$



La politique à évaluer avec

$\gamma = 1.0$ et $\alpha = 0.1$

Episode 2 :

$t=0$ (1, 1) NORD 0

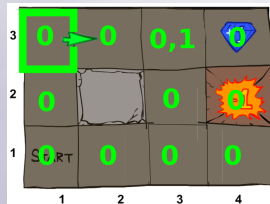
$t=1$ (1, 2) NORD 0

$t=2$ (1, 3) EST 0

$t=3$ (2, 3) EST 0

$t=4$ (3, 3) EST 0

$t=5$ (3, 2) NORD -1



Episode 2, $t = 2$,

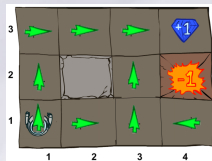
$s = \langle 1, 3 \rangle, s' = \langle 2, 3 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[0 + \gamma 0]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{\text{new_estim}} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{\text{old_estim}} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{\text{new_echantillon}}$$



Episode 2 :

t=0 (1, 1) NORD 0

t=1 (1, 2) NORD 0

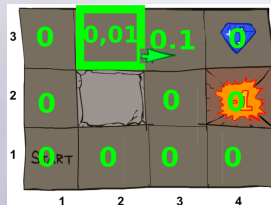
t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST 0

t=5 (3, 2) NORD -1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 2, $t = 3$,

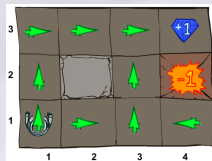
$s = \langle 2, 3 \rangle, s' = \langle 3, 3 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[0 + \gamma 0.1]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{\text{new_estim}} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{\text{old_estim}} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{\text{new_echantillon}}$$



Episode 2 :

t=0 (1, 1) NORD 0

t=1 (1, 2) NORD 0

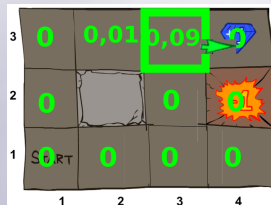
t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST 0

t=5 (3, 2) NORD -1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 2, $t = 4$,

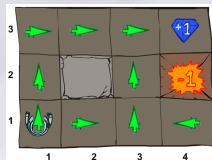
$s = \langle 3, 3 \rangle, s' = \langle 3, 2 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0.1 + \alpha[0 + \gamma 0]$$

AR passif : Algorithme TD Prediction : Exemple

Après **chaque action exécutée par l'agent** depuis un état s (étape $\langle s, a = \pi(s), s', r \rangle$), on applique la mise à jour suivante **à l'état s** :

$$\underbrace{V^\pi(s)}_{\text{new_estim}} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{\text{old_estim}} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{\text{new_echantillon}}$$



Episode 2 :

t=0 (1, 1) NORD 0

t=1 (1, 2) NORD 0

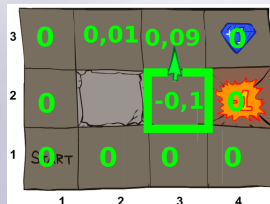
t=2 (1, 3) EST 0

t=3 (2, 3) EST 0

t=4 (3, 3) EST 0

t=5 (3, 2) NORD -1

La politique à évaluer avec $\gamma = 1.0$ et $\alpha = 0.1$



Episode 2, $t = 5$,

$s = \langle 3, 2 \rangle$, $s' = \langle 4, 2 \rangle$

$$V^\pi(s) \leftarrow (1 - \alpha)0 + \alpha[-1 + \gamma 0]$$

AR passif : algorithme 3 : TD Prediction

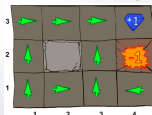
Algorithme 3 : TD Prediction

Intérêts/Inconvénients :

- ≡ + : apprentissage en ligne, à partir d'expériences, sans modèle
- ≡ ++ : pas besoin de conserver tout l'épisode, mise à jour à chaque étape
- ≡ + : convergence plus rapide avec le *bootstrap*
- ≡ + : faible variance avec le *bootstrap*
- ≡ - : biaisé : les valeurs ne sont pas toujours justes car on utilise le *bootstrap* pour l'échantillon ($V(s')$ peut être biaisé, surtout au début de l'apprentissage)

Résumons ...

AR passif : Tâche simplifiée d'évaluation d'une politique fixée



Pour une π fixée
(T et R inconnus)

0,8100	0,9000	1,0000	
0,7290		0,9000	
0,6561	0,7290	0,8100	0,7290

V^π

π fixée $\rightarrow$ Comment évaluer V^π ?

Pour chaque état s , on veut évaluer : $V^\pi(s) = E_\pi \{ \sum_{t=0}^{\infty} \gamma^t r_{t+1} | s_0 = s \}$

3 algorithmes dont TD Prediction qui met à jour un état s après chaque action dans cet état :

$$\underbrace{V^\pi(s)}_{\text{nouvelle_estim}} \leftarrow (1 - \alpha) \underbrace{V^\pi(s)}_{\text{ancienne_estim}} + \alpha \times \underbrace{(r + \gamma V^\pi(s'))}_{\text{nouvel_echantillon}}$$

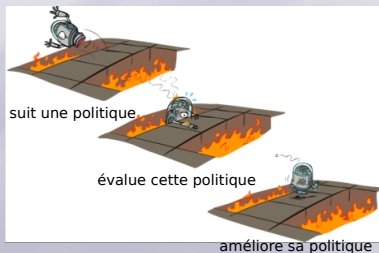
- ≡ Dans cette partie l'agent n'était pas actif ...
- ≡ Revenons à l'objectif initial : trouver π^* !

Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base
- 5 AR passif
- 6 AR actif**
- 7 Généralisation
 - Fusion d'états
 - Généralisation
 - Approximate Q-learning

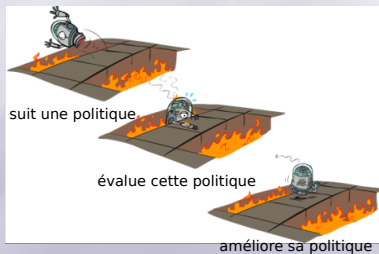
AR actif

- ≡ π n'est plus figée mais l'agent la modifie
- ≡ Objectif de l'agent : trouver la politique optimale π^* sans connaître T et R
 - L'agent suit une politique π
 - Il **évalue** cette politique : mise à jour de $V^\pi(s)$ selon TD prediction
 - Après chaque évaluation, il **améliore** la politique π suivie.



AR actif

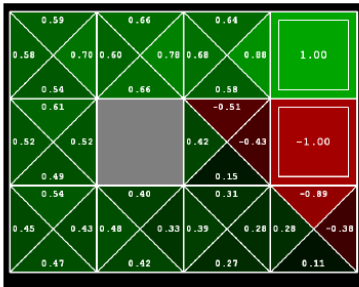
- ▢ π n'est plus figée mais l'agent la modifie
- ▢ Objectif de l'agent : trouver la politique optimale π^* sans connaître T et R
 - L'agent suit une politique π
 - Il évalue cette politique : mise à jour de $V^\pi(s)$ selon TD
 - Après chaque évaluation, il améliore la politique π suivie.



Comment améliorer une politique ?

Modifier les actions renvoyées par π qui ne sont plus les meilleures suite à la nouvelle évaluation

Fonction de valeur d'action Q



$$Q : S \times A \rightarrow \mathbb{R}$$

$Q(s, a)$ valeur de l'action a dans s

- $Q^\pi(s, a)$ évalue le retour espéré lorsque l'on effectue l'action a dans l'état s puis que l'on suit $\pi : Q^\pi(s, a) = E\{\sum_{t=0}^{\infty} \gamma^t r_{t+1} | \pi, s_t = s, a_t = a\}$
- Liens entre V et $Q : \forall s \in S \quad V(s) = \max_a Q(s, a)$

Comment améliorer une politique ? : Politique gloutonne

La (les) meilleure(s) action(s) dans s est (sont) celle(s) qui ont la valeur max dans $s : \pi(s) = \arg \max_{a \in A} Q(s, a) = \text{greedy}(Q(s, .))$

AR actif

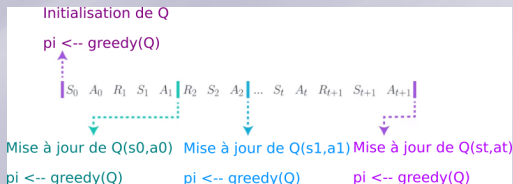
Agent glouton

A chaque étape $\langle s, a, s', r \rangle$:

- ≡ L'agent dans s suit sa politique $a = \pi(s)$
- ≡ Il **évalue** cette politique avec mise à jour TD : après chaque action, mise à jour de $Q(s, a)$ selon :

$$\underbrace{Q(s, a)}_{\text{nouvelle_estim}} \leftarrow (1 - \alpha) \underbrace{Q(s, a)}_{\text{ancienne_estim}} + \alpha \times \underbrace{(r + \gamma \max_b Q(s', b))}_{\text{nouvel_echantillon}}$$

- ≡ Après chaque mise à jour, il **améliore** sa politique :
- $$\pi(s) = \underset{a \in A}{\operatorname{greedy}}(Q(s, .)) = \arg \max_{a \in A} Q(s, a)$$



Apprentissage par renforcement actif

Agent glouton

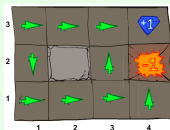
A chaque étape $\langle s, a, s', r \rangle$:

- ≡ L'agent dans s suit sa politique $a = \pi(s)$
- ≡ Il **évalue** cette politique avec mise à jour TD : après chaque action, mise à jour de $Q(s, a)$ selon :

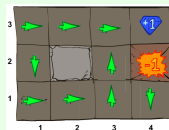
$$\underbrace{Q(s, a)}_{\text{nouvelle_estim}} \leftarrow (1 - \alpha) \underbrace{Q(s, a)}_{\text{ancienne_estim}} + \alpha \times \underbrace{(r + \gamma \max_b Q(s', b))}_{\text{nouvel_echantillon}}$$

- ≡ Après chaque mise à jour, il **améliore** sa politique :
- $$\pi(s) = \underset{a \in A}{\text{greedy}}(Q(s, .)) = \arg \max_{a \in A} Q(s, a)$$

Exemple



π initiale



π après 500 épisodes

Apprentissage par renforcement actif

Agent glouton

A chaque étape $\langle s, a, s', r \rangle$:

- ≡ L'agent dans s suit sa politique $a = \pi(s)$
- ≡ Il **évalue** cette politique avec mise à jour TD : après chaque action, mise à jour de $Q(s, a)$ selon :

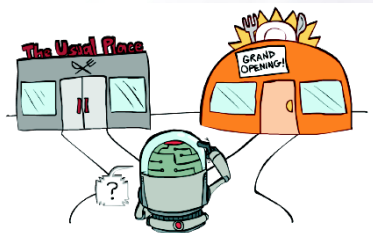
$$\underbrace{Q(s, a)}_{\text{nouvelle_estim}} \leftarrow (1 - \alpha) \underbrace{Q(s, a)}_{\text{ancienne_estim}} + \alpha \times \underbrace{\left(r + \gamma \max_b Q(s', b) \right)}_{\text{nouvel_echantillon}}$$

- ≡ Après chaque mise à jour, il **améliore** sa politique :

$$\pi(s) = \underset{a \in A}{\operatorname{greedy}}(Q(s, .)) = \arg \max_{a \in A} Q(s, a)$$

Approche limitée par le manque d'exploration.

Exploration et Exploitation



Trouver le compromis entre exploration et exploitation.

Exploitation / Glouton

Choisir la meilleure action en fonction de ce que l'on connaît jusqu'à présent (politique gloutonne / *greedy*)

Exploration

Essayer une action sous-optimale jusqu'à présent

→ gain de connaissance pour améliorer Q tant que Q^* n'a pas été trouvée : accélération de l'apprentissage

→ perte de temps si l'agent a déjà trouvé Q^*

Stratégies d'exploration

Politique ϵ -greedy

Explorer avec une probabilité ϵ :

$$\pi_{\epsilon\text{-greedy}}(s) =$$

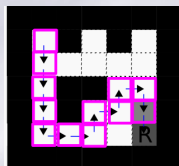
- ▮ action d'exploration (aléatoire) avec la probabilité ϵ
- ▮ action gloutonne ($a = \arg \max_a Q(s, a)$) avec la probabilité $1 - \epsilon$

ϵ -greedy traite toutes les actions, sauf la meilleure, de façon équivalente.

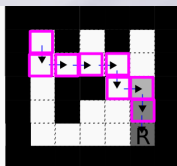
Algorithme du Q-learning

Construire une table des Q -valeurs (dimension $|S| \times |A|$) et répéter pour chaque épisode :

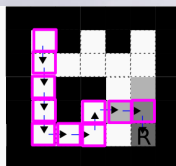
- ▢ $s \leftarrow$ état initial
- ▢ Répéter pour chaque étape dans l'épisode
 - Choisir a selon une stratégie d'exploration (**apprentissage actif**)
 - Exécuter a , observer r et s'
 - $Q(s, a) \leftarrow (1 - \alpha_k)Q(s, a) + \alpha_k[r + \gamma \max_b Q(s', b)]$ (**mise à jour TD**)
 - $s \leftarrow s'$



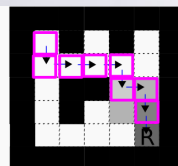
episode1



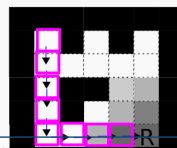
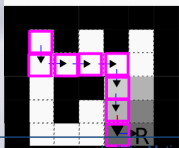
episode2



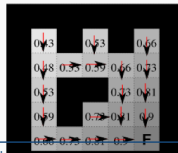
episode3



episode4



...

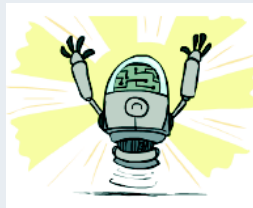


Algorithme du Q-learning

Propriétés du Q-learning

Q-learning converge vers Q^* si [Jaakkola,1994] :

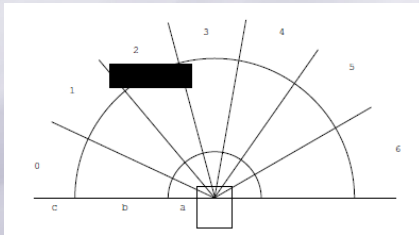
- ▢ S et A finis
- ▢ tous les couples (s, a) doivent être visités un nombre infini de fois
→ la stratégie d'exploration doit permettre assez d'exploration
- ▢ la suite $\alpha_k(s, a)$ doit décroître à chaque nouvelle visite de (s, a)
→ En pratique, α est choisi constant (adaptation permanente)



Exemple de mise en oeuvre

Apprendre à un robot à éviter des obstacles :

- ▢ télémètre laser et base mobile
- ▢ discrétisation de l'état : $|S| = 2187$
- ▢ 3 actions (avancer, tourner à droite, tourner à gauche)
- ▢ $r = -10$ si robot percute un obstacle (remis à son point de départ)
- ▢ $r = 3$ lorsque le robot choisit l'action d'avancer

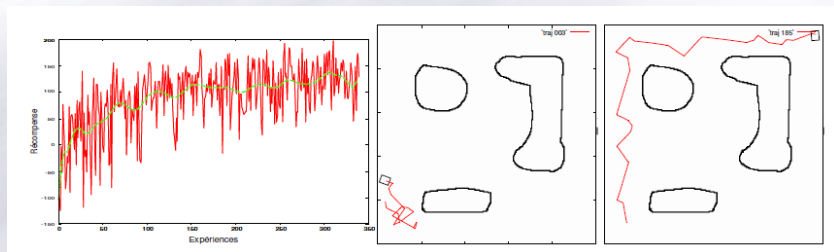


Discrétisation de l'espace d'états : 7 secteurs, pour chacun, O_i donne présence ou non d'obstacles dans les zones a et b ($O_i = 0$ si obstacle dans a , $O_i = 1$ si obstacle dans b , $O_i = 2$ sinon). L'état s est la valeur en base 3 fournie par les O_i : $s = \sum_k 3^k O_k$. Ici,

$$O_0 = O_1 = O_4 = O_5 = O_6 = 2, O_2 = O_3 = 1 \text{ donc } s = 2150.$$

Exemple de mise en oeuvre

Q-learning avec une table de taille 2187×3 .



épisode initial ($k = 0$) épisode $k = 185$

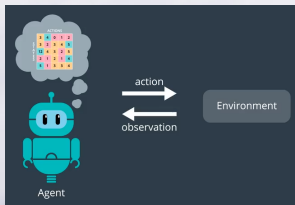
Un épisode correspond à 100 étapes (actions).

Exercice TD

Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base
- 5 AR passif
- 6 AR actif
- 7 Généralisation
 - Fusion d'états
 - Généralisation
 - Approximate Q-learning

Limites des approches tabulaires



Les algorithmes présentés jusqu'ici sont des algorithmes **tabulaires**

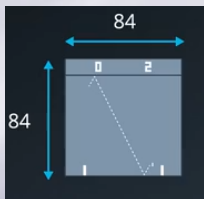
- utilisation d'une table pour stocker les Q -valeurs de chaque état-action (s, a)
- hypothèse d'espaces d'états et d'actions discrets et finis

Limites des approches tabulaires

- utilisation d'une table pour stocker les Q -valeurs
- chaque expérience (s, a, s', r) à l'instant t ne permet d'apprendre que pour le couple état-action (s, a) correspondant à cette expérience.

Problème 1 : espace mémoire

- Besoin de stocker $|S| \times |A|$ éléments.



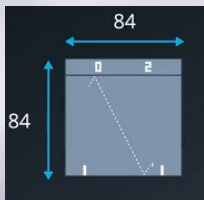
Par ex. si les états sont des images de taille 84×84 , en niveau de gris (256 valeurs) : $|S| = 256^{84 \times 84} = 256^{7056}$

Limites des approches tabulaires

- utilisation d'une table pour stocker les Q -valeurs
- chaque expérience (s, a, s', r) à l'instant t ne permet d'apprendre que pour le couple état-action (s, a) correspondant à cette expérience.

Problème 1 : espace mémoire

- Besoin de stocker $|S| \times |A|$ éléments.



Par ex. si les états sont des images de taille 84×84 , en niveau de gris (256 valeurs) : $|S| = 256^{84 \times 84} = 256^{7056}$

Problème 2 : Lenteur de l'apprentissage

- il faut visiter tous les états un grand nombre de fois
- pas de **généralisation** de l'expérience apprise à des états jamais rencontrés



Si les états du MDP sont positions 2D des éléments (pacman, fantômes, dots, ...). Si l'agent apprend que l'état à gauche est mauvais, il ne saura rien sur les états à droite s'il ne les a jamais rencontrés...

Limites des approches tabulaires

- ▮ utilisation d'une table pour stocker les Q -valeurs
- ▮ chaque expérience (s, a, s', r) à l'instant t ne permet d'apprendre que pour le couple état-action (s, a) correspondant à cette expérience.

Problème 1 : espace mémoire

- ▮ Besoin de stocker $|S| \times |A|$ éléments.

Problème 2 : Lenteur de l'apprentissage

- ▮ il faut visiter tous les états un grand nombre de fois
- ▮ pas de **généralisation** de l'expérience apprise à des états jamais rencontrés

- ▮ hypothèse d'espaces d'états **discrets** et finis

Problème 3 : Discrétisation

- ▮ Comment choisir une discrétisation adaptée ?

Quelles solutions à ces 3 problèmes ?

Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base
- 5 AR passif
- 6 AR actif
- 7 Généralisation
 - Fusion d'états
 - Généralisation
 - Approximate Q-learning

Solution 1 : Fusion d'états

Fusion d'états

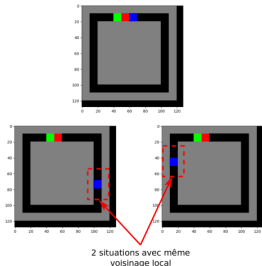
- ≡ Si des **situations se ressemblent**, on considère qu'elles correspondent au **même état discret**
- ≡ Recherche de **regroupements** des situations similaires
 - Experte
 - Voisinage local identique
 - Symétrie
 - Clustering selon fonction de similarité
 - Discrétisation de l'espace des états (pour le cas continu)
- ≡ Le MDP dans lequel on évolue est alors simplifié car on a **moins d'états** suite à la fusion !

Solution 1 : Fusion d'états discrets

- Objectifs : appliquer un Q-learning tabulaire
- Contexte : états bruts discret
- Problème : états bruts trop nombreux → limitation de mémoire et/ou apprentissage lent
- Solution : fusion d'états

Exemple

Les états sont des images → Trop d'états → On fusionne des situations similaires pour réduire $|S|$

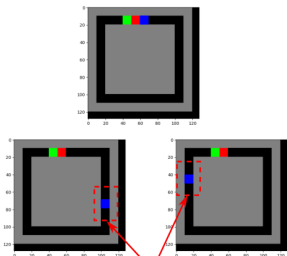


Faut-il les considérer comme un même état du MDP ?

Solution 1 : Fusion d'états discrets

Difficultés de la fusion d'états

La politique définie sur l'espace des **états fusionnés** doit permettre à l'agent de prendre une décision markovienne, i.e. qui ne dépend que de l'état courant.



2 situations avec même
voisinage local

Si ces 2 situations sont fusionnées, elles correspondent à un même état du MDP, l'action optimale dans ces situations doit être la même ($\pi : S \rightarrow A$) !

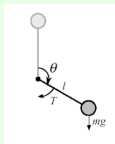
Difficultés de la fusion d'états

Quelles informations conserver pour définir les états du MDP à partir des informations brutes ? Quelles informations sont **nécessaires et suffisantes** pour décider d'une action optimale pour chaque état ?

Solution 1 : Fusion d'états continus

- Objectifs : appliquer un Q-learning tabulaire
- Contexte : états bruts continus
- Problème : Comment choisir la discrétisation des états continus ?
- Solution : fusion d'états avec discrétisation

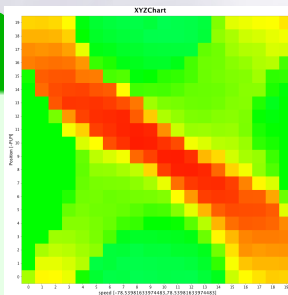
Exemple du Pendule inversé (video)



Espace d'états continu (2D) :

$$\mathbf{s} = (\theta, \dot{\theta})$$

9 actions (*torque*) ; $R(\mathbf{s}) = \cos(\theta)$



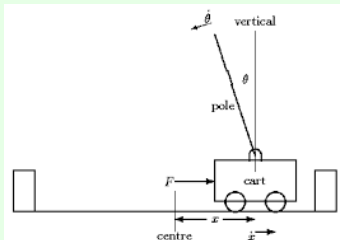
$V(s)$

Q-learning tabulaire fonctionne avec une discrétisation uniforme de S (30×30)

Solution 1 : Fusion d'états continus

- ≡ Objectifs : appliquer un Q-learning tabulaire
- ≡ Contexte : états bruts continus
- ≡ Problème : Comment choisir la discrétisation des états continus ?
- ≡ Solution : fusion d'états avec discrétisation

Exemple : Pendule inversé sur chariot (video)



Espace d'états continu (4D) :

$$\mathbf{s} = (x, \dot{x}, \theta, \dot{\theta})$$

3 actions : $a = \{-F, 0, F\}$

$R(\mathbf{s}) = \cos(\theta) - d$ avec d pénalité si proche du bord

Q-learning fonctionne uniquement avec **discrétisation non-uniforme** de S :

$$\equiv x = \{-0.8, 0.8\}; \quad \dot{x} = \{-0.5, 0.5\}$$

$$\equiv \theta = \{-0.105, -0.017, 0, 0.017, 0.105\}; \quad \dot{\theta} = \{-0.873, 0, 0.873\}$$

Solution 1 : Fusion d'états continus

Comment choisir la discrétisation de S ?

Discrétisation de l'espace d'états experte, difficile à trouver :

- ▮ si trop détaillée, la discrétisation sera computationnellement ingérable (trop d'états → problème de mémoire / lenteur d'apprentissage)
- ▮ si trop faible, la discrétisation pourrait conduire à une politique non pertinente (perte de la propriété de Markov)

Solution 1 : Fusion d'états

Conclusion

- méthode pour réduire le nombre d'états si beaucoup d'états
- méthode pour vérifier l'hypothèse d'états discrets

On résoud les problèmes d'espace mémoire et de lenteur d'apprentissage partiellement et selon les cas.

Limitations

Complexe à réaliser dans certains cas :

- besoin de connaissance expertes sur le problème
- impossible si environnement très complexe
- risque de définir des états fusionnés non adaptés, i.e. pour lesquels la politique ne sera pas pertinente (ne permettra pas de prendre une décision markovienne)

Ne permet pas a généralisation.

Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base
- 5 AR passif
- 6 AR actif
- 7 **Généralisation**
 - Fusion d'états
 - **Généralisation**
 - Approximate Q-learning

Solution 2 : Généralisation entre états

Exemple Jeu Pac-Man

On décrit l'état du MDP avec les positions 2D des éléments (pacman, fantômes, dots, ...).

L'agent apprend
par l'expérience
que cet état est mauvais



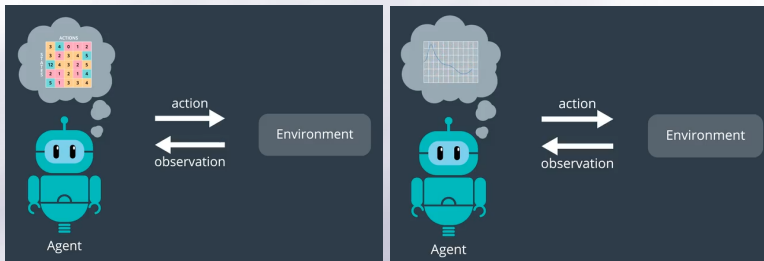
Avec le Q-learning
tabulaire, l'agent
ne saura rien sur cet état !



Ou même sur
celui-ci !



Solution 2 : Généralisation entre états

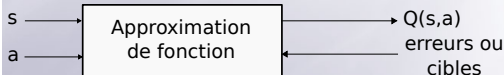
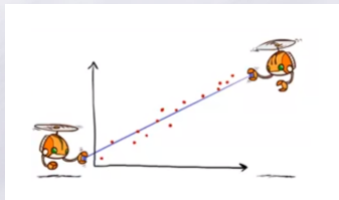


- Objectif : généraliser/étendre les expériences apprises à de nouvelles situations jamais rencontrées.
- Principe : utiliser une **fonction de Q -valeurs continue** plutôt qu'une table

Solution 2 : Généralisation entre états

Approximation de fonction

Problème classique en *Machine Learning*



$Q(s, a) =$

- ≡ table
- ≡ **Approximation linéaire et vecteur de *features* expertes** : Approximate Q-learning
- ≡ Approximation non-linéaire : réseau de neurones (cf. CM3)

Plan

- 1 Différents types d'apprentissage
- 2 Historique de l'AR
- 3 Rappels planification
- 4 Notions de base
- 5 AR passif
- 6 AR actif
- 7 **Généralisation**
 - Fusion d'états
 - Généralisation
 - **Approximate Q-learning**

Approximate Q-learning

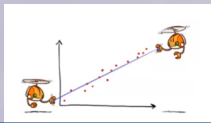
La fonction de valeur Q est approximée par une fonction linéaire paramétrée par des poids w_i :

$$Q_w(s, a) = \sum_{i=1}^n w_i f_i(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

- ≡ n *features* ou fonctions caractéristiques $f_i : S \times A \rightarrow R$ **choisies** (expertes)
- ≡ n paramètres/poids w_i **appris** pour avoir la meilleure **approximation de la fonction Q**

Avantages

- ≡ on stocke n poids au lieu des Q valeurs ($n \ll |S| \times |A|$)
- ≡ apprentissage conjoint des Q-valeurs (généralisation) : pour un état s jamais rencontré, on a une valeur $Q_w(s, a)$ peut être pas mauvaise



Approximate Q-learning : choix des features

La fonction de valeur Q est approximée par une fonction linéaire paramétrée par des poids w_i :

$$Q_w(s, a) = \sum_{i=1}^n w_i f_i(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

Features $f_i : S \times A \rightarrow R$

Description d'un état avec un vecteur de n features :

- ▢ une feature f_i associe une valeur à tout état ou état-action
- ▢ une **feature** f_i **capture une ou plusieurs propriétés importantes d'un état**
- ▢ information souvent incomplète, fixée manuellement

Approximate Q-learning : choix des features

La fonction de valeur Q est approximée par une fonction linéaire paramétrée par des poids w_i :

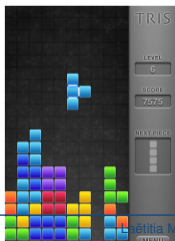
$$Q_w(s, a) = \sum_{i=1}^n w_i f_i(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

Features $f_i : S \times A \rightarrow R$

Description d'un état avec un vecteur de n features :

- une feature f_i associe une valeur à tout état ou état-action
- une **feature f_i capture une ou plusieurs propriétés importantes d'un état**
- information souvent incomplète, fixée manuellement

Exemple du Tetris



Les différentes features peuvent être :

- pièce en cours
- hauteur moyenne
- nombre de trous
- nombre de briques supprimées

...

Approximate Q-learning : apprentissage des poids

La fonction de valeur Q est approximée par une fonction linéaire paramétrée par des poids w_i :

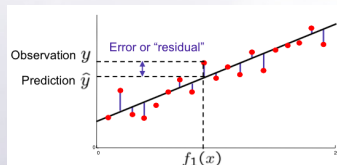
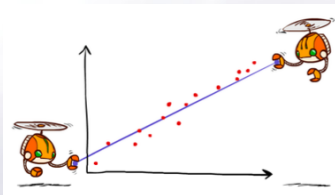
$$Q_w(s, a) = \sum_{i=1}^n w_i f_i(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

≡ n paramètres/poids w_i **appris** pour avoir la meilleure approximation de la fonction Q

Comment apprendre les poids ?

Apprentissage supervisé !

Rappels : régression linéaire 1D



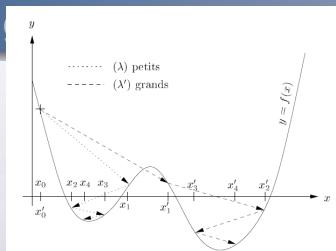
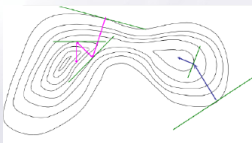
- = f_w fonction cible (**modèle**) à apprendre ($y = f_w(x) = w_0 + w_1 x$)
- = N données d'entrée labellisées/**échantillons** : (x_k, y_k)
- = prédictions $\hat{y}_k = w_0 + w_1 * x_k$

- = Objectif : Trouver $\vec{w}$ qui minimise l'erreur entre prédictions et valeurs cibles :

$$E(\vec{w}) = 1/N \sum_k (y_k - \hat{y}_k)^2$$

- = Moyen : descente de gradient

Rappels : Descente de



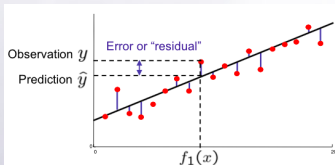
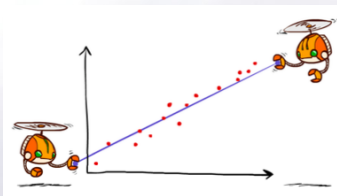
Algorithme itératif pour trouver le minimum local d'une fonction

- ≡ A t on part du point $\vec{x}_t$
- ≡ Le gradient de f , noté $\vec{\nabla} f(\vec{x}_t)$, indique la **direction de plus grande pente** de f
- ≡ Le gradient de f est le **vecteur des dérivées partielles** de f par rapport à ses **paramètres** :

$$\vec{\nabla} f(x_1, \dots, x_K) = \left(\frac{\partial f(x_1, \dots, x_K)}{\partial x_1}, \frac{\partial f(x_1, \dots, x_K)}{\partial x_2}, \dots \right)^T$$

- ≡ On suit le vecteur donné par le gradient de f en $\vec{x}_t$
- ≡ On **met à jour les paramètres** : $\vec{x}_{t+1} = \vec{x}_t - \lambda \vec{\nabla} f(\vec{x}_t)$; $\lambda > 0$ pas d'apprentissage permet de progresser rapidement au début (en sortant si possible de zones d'optima locaux) puis d'affiner le résultat par des pas plus petits.

Rappels : régression linéaire 1D



A partir d'un ensemble d'échantillons (x_k, y_k) :

1. Calcul des prédictions

$$\hat{y}_k = w_0 + w_1 x_k$$

2. Calcul de l'erreur

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

≡

$$\frac{\partial MSE}{\partial w_1} = \frac{\partial MSE}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_1} = -2 \frac{1}{N} \sum_{i=1}^N x_i (y_i - \hat{y}_i)$$

≡

4- Mise à jour des **poids** (une itération de la descente de gradient) $\alpha \in [0; 1]$ learning rate :

3. Calcul du gradient de l'erreur :

$$\frac{\partial MSE}{\partial w_0} = \frac{\partial MSE}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial w_0} = -2 \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)$$

$$w_0 = w_0 - \alpha \frac{\partial MSE}{\partial w_0}$$

$$w_1 = w_1 - \alpha \frac{\partial MSE}{\partial w_1}$$

Approximate Q-learning : apprentissage des poids

Approximation linéaire de la Q fonction :

$$Q_w(s, a) = \sum_{i=1}^n w_i f_i(s, a) = w_1 f_1(s, a) + w_2 f_2(s, a) + \dots + w_n f_n(s, a)$$

≡ n paramètres/poids w_i **appris** pour avoir la meilleure **approximation de la fonction Q**

On veut utiliser la descente de gradient pour trouver les poids qui minimisent l'erreur entre prédictions et valeurs cibles.

Quelles sont ici les valeurs cibles ?

Apprentissage supervisé (régression) : à chaque étape (s, a, s', r) :

- ≡ la prédiction pour la valeur de (s, a) est $\hat{y} = Q_w(s, a) = \sum_i w_i f_i(s, a)$
- ≡ la valeur cible de (s, a) est estimée par **amorçage** (*bootstrap*) (cf. TD prediction) :
 - $Q(s, a)$ évalue la somme pondérée des récompenses espérées depuis (s, a) soit

$$r + \gamma \underbrace{r(s', a', s'') + \gamma^2 r(s'', a'', s''') + \dots}_{\rho \text{ somme des recompenses esperees depuis } s'}$$

- $V(s') = \max_b Q(s', b)$ est une estimation de la somme pondérée des récompenses depuis $s' \rightarrow$ on peut estimer ρ avec $V(s')$
- Ainsi la valeur cible $y = Q(s, a)$ peut être estimée par

$$y = r + \gamma \max_{b \in A} Q_w(s', b)$$

Approximate Q-learning : apprentissage des poids

Exercice

Quelle est l'équation de mise à jour des poids w à chaque étape (s, a, s', r) ?

Pour une étape (s, a, s', r) :

1. Calcul de la prédiction $\hat{y} = Q_w(s, a) = \sum_i w_i f_i(s, a)$
2. Calcul de l'erreur entre prédiction $\hat{y}$ et valeur cible y

$$E(\vec{w}) = 1/2(y - \hat{y})^2$$

avec estimation de la valeur cible $y = r + \gamma \max_{b \in A} Q_w(s', b)$

3. Calcul du gradient de l'erreur (on suppose y indépendant de w !) : dérivées partielles de E par rapport à ses paramètres (rappel : $(u^2)' = 2uu'$)

$$\frac{\partial E(\vec{w})}{\partial w_m} = ?$$

4. Mise à jour de chaque paramètre w_m :

$$w_m = ?$$

Approximate Q-learning : apprentissage des poids

Pour une étape (s, a, s', r) :

1. Calcul de la prédiction $\hat{y} = Q_w(s, a) = \sum_i w_i f_i(s, a)$
2. Calcul de l'erreur entre prédiction $\hat{y}$ et valeur cible y

$$E(\vec{w}) = 1/2(y - \hat{y})^2 = 1/2(y - \sum_i w_i f_i(s, a))^2$$

3. Calcul du gradient de l'erreur (on suppose y indépendant de w !) :

$$\frac{\partial E(\vec{w})}{\partial w_m} = (y - \hat{y}) \frac{\partial (y - \sum_i w_i f_i(s, a))}{\partial w_m} = (y - \sum_i w_i f_i(s, a))(-f_m(s, a))$$

4. Mise à jour de chaque paramètre w_m ($\forall m \in [1, n]$) :

$$w_m \leftarrow w_m - \alpha \frac{\partial E(\vec{w})}{\partial w_m}$$

$$w_m \leftarrow w_m + \alpha(y - \hat{y})f_m(s, a)$$

$$w_m \leftarrow w_m + \alpha(r + \gamma \max_b Q_w(s', b) - Q_w(s, a))f_m(s, a)$$

Interprétation : si mauvaise prédiction, ajustement des poids.

Approximate Q-learning

Exercice 2 du TD.